

From Vibe Coding to Spec-Driven Development

How SDD is re-emerging as a critical methodology for AI-assisted development

Cuong Kane

cuongkane.com

April 2026

- 1 Context
- 2 Problem Statement
- 3 SDD Solution
- 4 SDD Workflow
- 5 Adopting SDD in Practice
- 6 Conclusion

AI Has Changed How We Code

In 2026, AI coding tools are part of every developer's daily workflow.

- “**Vibe coding**” has entered the mainstream – prompt and let the AI handle the rest
- LLMs can now implement most technical tasks **autonomously**
- The developer's role has shifted: less writing code, more **directing and reviewing**

The bottleneck is no longer typing speed – it's clarity of intent.

The New Developer Role

AI is great at

- *How* to build something
- Generating code fast
- Following patterns
- Repetitive tasks

Humans still own

- *What* to build and *why*
- Defining requirements
- Validating correctness
- Business judgment

- 1 Context
- 2 Problem Statement
- 3 SDD Solution
- 4 SDD Workflow
- 5 Adopting SDD in Practice
- 6 Conclusion

The Hidden Cost of AI Speed

Teams relying heavily on AI coding agents consistently hit four problems:

1. Quality at scale

Code generated faster than teams can review

2. Redundant loops

Costly prompt-review-correct cycles

3. Token consumption

AI ingests too much context, diminishing returns

4. Code is a liar

AI replicates bad patterns from accidental behavior

Quality at Scale

- AI produces hundreds of lines in minutes
- Verifying correctness still requires **human judgment**
- Without a shared spec, code review becomes a **bottleneck** – or a rubber stamp

Speed without direction creates technical debt faster than ever.

The Redundant Loop

Without a clear spec, developers fall into a costly refinement cycle:

- ① **Prompt:** “create a button in the center”
- ② **AI:** creates it too big with ugly centering
- ③ **Prompt:** “no, I need...”
- ④ Repeat **dozens of times** per feature

A clear spec upfront eliminates most of these rounds.

Token Waste & Code as a Liar

Token Consumption

- AI reads entire repo to understand behavior
- Codebases are poorly structured for machines
- Heavy token use, diminishing returns

Code is a Liar

- Dead code, redundant complexity
- AI hallucinates intent from accidental behavior
- Specs capture *intended*, not *accidental*

- 1 Context
- 2 Problem Statement
- 3 SDD Solution
- 4 SDD Workflow
- 5 Adopting SDD in Practice
- 6 Conclusion

What is Spec-Driven Development?

- **SDD** = write a detailed requirement specification *before* writing code
- Not new – Microsoft introduced it in the early 2000s
- Most familiar example: **API-first development** (OpenAPI spec as contract)

In 2026, SDD is re-emerging because the developer's primary job has shifted from writing code to writing and clarifying requirements.

Level 1: Spec-First

- Spec is written **before** implementation to guide initial development
- Once code exists, spec may or may not be maintained
- Best for: **prototypes, POCs**, projects that won't be maintained long-term

Forces upfront clarity, even if the spec becomes disposable.

Level 2: Spec-Anchored

- Spec is maintained **alongside the code** throughout the system's lifecycle
- Changes to behavior require updating **both** spec and code
- AI reads the spec instead of reverse-engineering intent from the codebase
- Tests follow the spec strictly and validate against source code

Best fit for most modern software teams using AI agents.

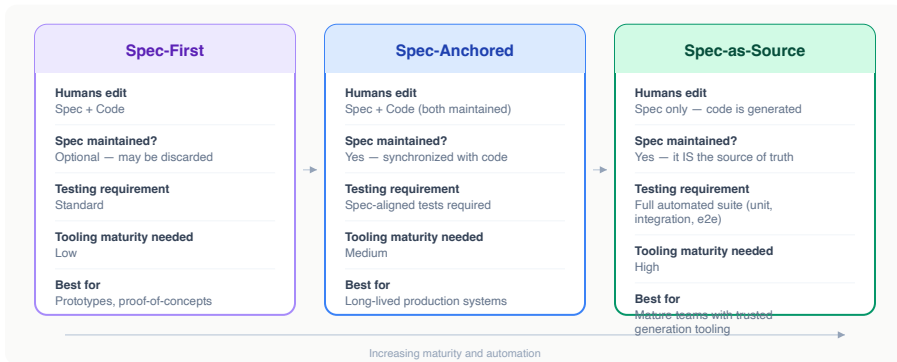
Level 3: Spec-as-Source

- Spec is the **only artifact** humans edit directly
- Code is entirely **generated** from the spec – never manually modified
- Requires mature code generation tooling and comprehensive automated tests

Spec becomes the single source of truth; codebase stays deterministic.

A higher level doesn't always mean better. Start where your team is ready.

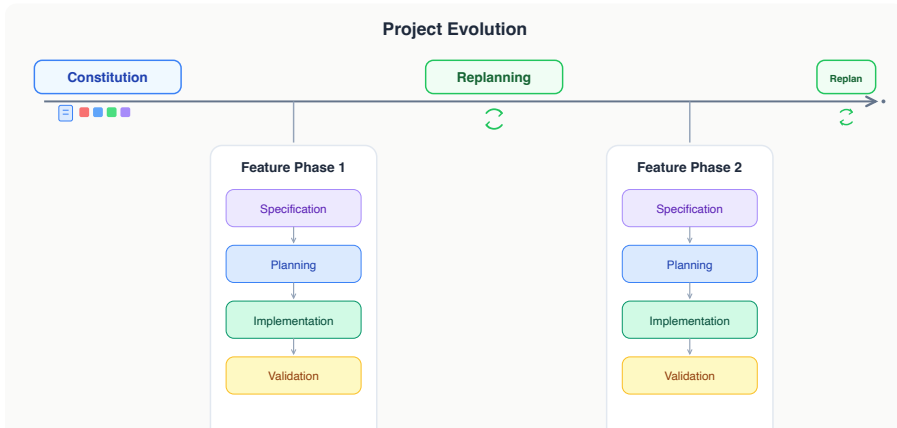
Three Levels of SDD



- 1 Context
- 2 Problem Statement
- 3 SDD Solution
- 4 SDD Workflow**
- 5 Adopting SDD in Practice
- 6 Conclusion

A Typical SDD Workflow

Three phases: **Constitution**, **Feature Development**, and **Replanning**.



Phase 1: Constitution

Define the high-level foundation for your project:

Greenfield

- Project goal
- Tech stack
- Roadmap

Brownfield

- Read existing codebase
- Build initial specification
- Capture project purpose

Outcome: living documentation as the entry point for humans and AI.

Phase 2: Feature Development

Each new feature follows four steps:

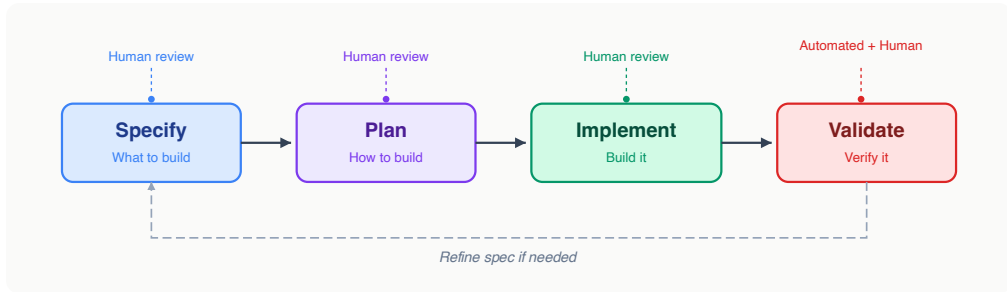
- ❶ **Specification** – clarify requirements, use Gherkin format for acceptance criteria
- ❷ **Planning** – write implementation plan (architecture, boundaries, data flow)
- ❸ **Implementation** – break into smaller tasks, feed to AI agent with tests
- ❹ **Validation** – run tests; if they fail, fix code or correct spec (human decision)

Phase 3: Replanning

- No plan survives first contact with code
- **Replanning** = updating the spec to reflect what was learned
- Keeps spec and code synchronized rather than drifting apart
- Can be triggered by AI, but always requires **human approval**

This feedback loop is also known as “harness engineering.”

The Human Factor



Replanning happens after code review when requirements change – always with **human approval**.

- 1 Context
- 2 Problem Statement
- 3 SDD Solution
- 4 SDD Workflow
- 5 Adopting SDD in Practice
- 6 Conclusion

SDD Tools Landscape (April 2026)

Tool	Best fit	Complexity	Workflow style
Spec Kit	Structured checkpoints	Medium	specify → plan → implement
BMAD	Enterprise, multi-role	High	Multi-agent personas
OpenSpec	Brownfield, fast iteration	Low	propose → apply → archive
Kiro	AWS-native, IDE integration	Medium	Integrated IDE pipeline

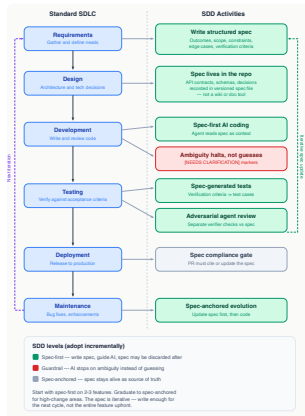
- Spec Kit, BMAD, OpenSpec are **agent-agnostic** (Claude, Copilot, Cursor, etc.)
- Kiro is tied to AWS Bedrock but offers the most integrated experience

SDD Reshapes the SDLC

SDD doesn't replace the SDLC – it reshapes it:

- Requirements → spec authoring
- Design → planning with AI
- Implementation → AI-driven generation
- QA → automated spec-based validation

Specs are the connective tissue between every phase.



- 1 Context
- 2 Problem Statement
- 3 SDD Solution
- 4 SDD Workflow
- 5 Adopting SDD in Practice
- 6 Conclusion

Key Takeaways

- Use **spec-first** for initial development, **spec-anchored** for production systems, **spec-as-source** only when generation tooling is mature
- Traditional design docs are advisory; **SDD specs are enforced** – tests fail if code diverges
- Like TDD and BDD, SDD focuses on **end-user behavior and original intent** before writing code
- Adopting SDD requires setup effort, good automated tests, and a mindset shift to treat the spec as a **first-class artifact**

SDD is a sustainable investment for teams building with AI.

References

- Spec-Driven Development: From Code to Contract – Deepak Babu Piskala
- Understanding SDD: Kiro, spec-kit, and Tessel – Martin Fowler
- Deep learning AI – AI Agentic Workflows
- GitHub Spec Kit | OpenSpec | BMAD Method | Kiro

Thank You